

UNIT III COMPUTER FUNDAMENTALS

Functional Units of a Digital Computer: Von Neumann Architecture – Operation and Operands of Computer Hardware Instruction – Instruction Set Architecture (ISA): Memory Location, Address and Operation – Instruction and Instruction Sequencing – Addressing Modes.

Functional Units of a Digital Computer

A computer consists of five functionally independent main parts. They are

1. Input
2. Memory
3. Arithmetic and logic unit
4. Output
5. Control unit

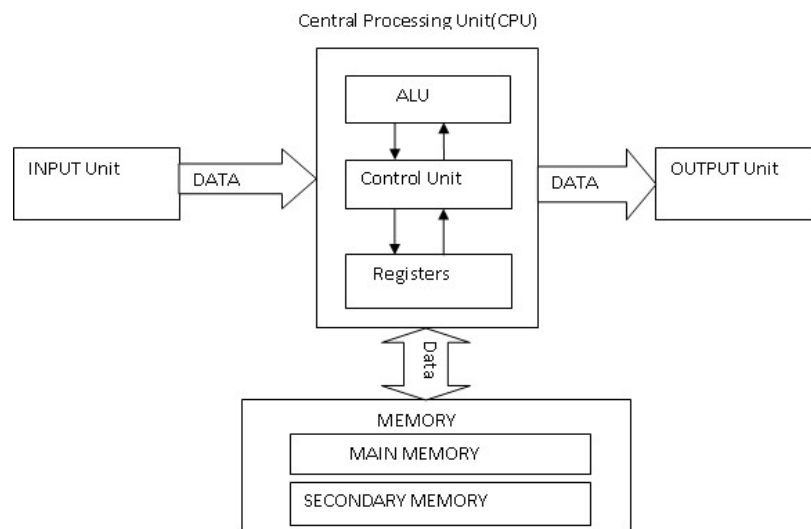


Fig 3.1 The basic components of computer system

The computer accepts programs and the data through an input and stores them in the memory. The stored data are processed by the arithmetic and logic unit under program control. The processed data is delivered through the output unit. All above activities are directed by control unit.

a. Input unit

The computer accepts coded information through input unit. The input can be from human operators; electromechanical devices such as keyboards or from other computer over communication lines.

Examples of input devices are Keyboard, joysticks; trackballs and mouse are used as graphic input devices in conjunction with display.

- It is a common input device.
- Whenever a key is pressed; the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted over cable to the memory of the computer.

b. Memory unit

Memory unit is used to store programs as well as data. Memory is classified into primary and secondary storage.

Primary Storage

It also called main memory. It operates at high speed and it is expensive. It is made up of large number of semiconductor storage cells, each capable of storing one bit of information. These cells are grouped together in a fixed size called word. This facilitates reading and writing the content of one word (n bits) in single basic operation instead of reading and writing one bit for each operation.

Secondary Storage

It is slow in speed. It is cheaper than primary memory. Its capacity is high. It is used to store information that is not accessed frequently. Various secondary devices are magnetic tapes and disks, optical disks (CD-ROMs), floppy etc.

c. Arithmetic and Logic Unit

Arithmetic and logic unit (ALU) and control unit together form a processor. Actual execution of most computer operations takes place in arithmetic and logic unit of the processor. Example: Suppose two numbers located in the memory are to be added. They are brought into the processor, and the actual addition is carried out by the ALU.

Registers are high speed storage elements available in the processor. Each register can store one word of data. When operands are brought into the processor for any operation, they are stored in the registers. Accessing data from register is faster than that of the memory.

d. Output unit

The function of output unit is to produce processed result to the outside world in human understandable form. Examples of output devices are Graphical display, Printers such as inkjet, laser, dot matrix and so on. The laser printer works faster.

e. Control unit

Control unit coordinates the operation of memory, arithmetic and logic unit, input unit, and output unit in some proper way. The **control unit** issues **control signals** that cause the CPU (and other components of the computer) to fetch the instruction to the IR (Instruction Register) and then execute the actions dictated by the machine language instruction that has been stored there. Control units are well defined, physically separate unit that interact with other parts of the machine. A set of control lines carries the signals used for timing and synchronization of events in all units. Example: Data transfers between the processor and the memory are controlled by the control unit through timing signals. **Timing signals** are the signals that determine when a given action is to take place.

Computer Components:

Top-Level view:

- **program counter (PC)** contains the address of the assembly language instruction to be executed next.
- **Instruction Register (IR)** contains the binary word corresponding to the machine language version of the instruction currently being executed.
- **Memory Address Register (MAR)** contains the address of the word in main memory that is being accessed. The word being addressed contains either data or a machine language instruction to be executed.
- **Memory Buffer Register (MBR)** also called **MDR** for memory data register) is the register used to communicate data to and from the memory.

Basic Operational Concept

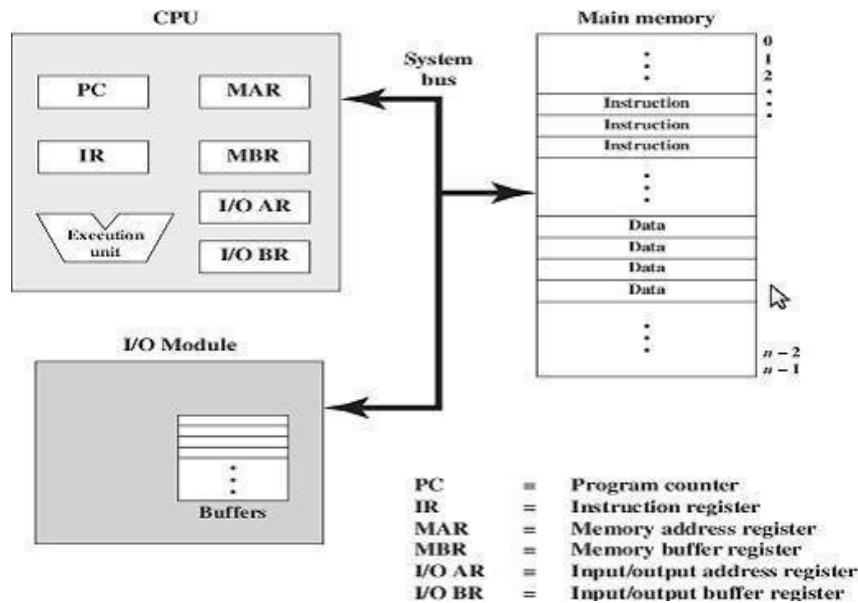


Fig 3.2 Computer Components

Fetch – Decode – Execute Cycle:

The operation of a processor is characterized by a fetch-decode- execute cycle.

1. **Fetch:** In the first phase of the cycle, the processor fetches an instruction from memory. The address of the instruction to fetch is stored in an internal register named the program counter, or PC. As the processor is waiting for the memory to respond with the instruction, it increments the PC. This means the fetch phase of the next cycle will fetch the instruction in the next sequential location in memory.
2. **Decode:** In the decode phase the processor stores the information returned by the memory in another internal register, known as the as the instruction register, or IR. The IR now holds a single machine instruction encoded as a binary number. The processor decodes the value in the IR in order to figure out which operations to perform in the next stage
3. **Execute:** In the execution stage the processor actually carries out the instruction. This step often requires further memory operations; for example, the instruction may direct the processor to fetch two operands from memory, add them, and store the result in a third location (the addresses of the operands and the result are also encoded as part of the instruction). At the end of this phase the machine starts the cycle over again by entering the fetch phase for the next instruction. The CPU exchanges data with

memory.

The CPU exchanges data with memory. For this purpose, it typically makes use of **two internal (to the CPU) register**:

- A memory address register (MAR), which specifies the address in memory for the next read or write, and
- A memory buffer register (MBR), which contains the data to be written into memory or receives the data read from memory.

An I/O addresses register (I/OAR) specifies a particular I/O device.

An I/O buffer (I/OBR) register is used for the exchange of data between an I/O module and the CPU. A memory module consists of a set of locations, defined by sequentially numbered address. Each location contains a binary number that can be interpreted as either an instruction or data. An I/O module transfers data from external devices to CPU and memory, and vice versa.

It contains internal buffers for temporarily holding these data until they can be sent on. Instructions can be classified as one of three major types: arithmetic/logic, data transfer, and control. Arithmetic and logic instructions apply primitive functions of one or two arguments, for example addition, multiplication, or logical AND.

Von Neumann Architecture

Von Neumann architecture was first published by John von Neumann in 1945. His computer architecture design consists of a Control Unit, Arithmetic and Logic Unit (ALU), Memory Unit, Registers and Inputs/Outputs. It is based on the stored-program computer concept, where instruction data and program data are stored in the same memory. This design is still used in most computers produced today.

Central Processing Unit (CPU)

The Central Processing Unit (CPU) is the electronic circuit responsible for executing the instructions of a computer program. It is sometimes referred to as the microprocessor or processor. The CPU contains the ALU, CU and a variety of registers.

Registers

Registers are high speed storage areas in the CPU. All data must be stored in a register before it can be processed.

Arithmetic and Logic Unit (ALU)

The ALU allows arithmetic (add, subtract etc.) and logic (AND, OR, NOT etc.) operations to be carried out.

Control Unit (CU)

The control unit controls the operation of the computer's ALU, memory and input/output devices, telling them how to respond to the program instructions it has just read and interpreted from the memory

unit. The control unit also provides the timing and control signals required by other computer components.

Buses

Buses are the means by which data is transmitted from one part of a computer to another, connecting all major internal components to the CPU and memory.

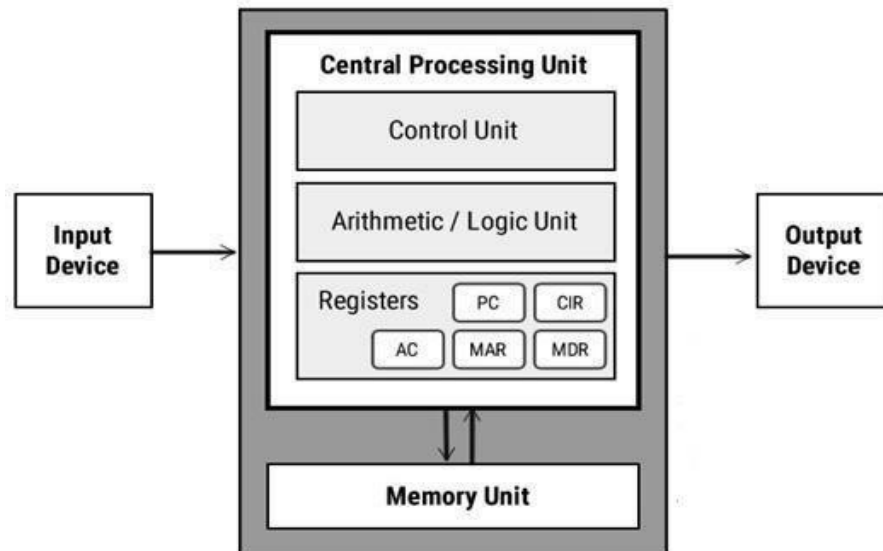


Fig 3.3 Von Neumann architecture

MAR	Memory Address Register	Holds the memory location of data that needs to be accessed
MDR	Memory Data Register	Holds data that is being transferred to or from memory
AC	Accumulator	Where intermediate arithmetic and logic results are stored
PC	Program Counter	Contains the address of the next instruction to be executed
CIR	Current Instruction Register	Contains the current instruction during processing

A standard CPU system bus is comprised of a control bus, data bus and address bus.

Address Bus	Carries the addresses of data (but not the data) between the processor and memory
Data Bus	Carries the data from input/to output devices
Control Bus	Carries control signals/commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer

Memory Unit

The Memory unit consists of RAM, sometimes referred to as primary or main memory. Unlike a hard drive (secondary memory), this memory is fast and also directly accessible by the CPU. RAM is split into partitions: Each partition consists of an address and its contents (both in binary form). The address will uniquely identify every location in the memory. Loading data from permanent memory (hard drive), into the faster and directly accessible temporary memory (RAM), allows the CPU to operate much quicker.

Von Neumann bottleneck

Whatever we do to enhance performance, we cannot get away from the fact that instructions can only be done one at a time and can only be carried out sequentially. Both of these factors hold back the competence of the CPU. This is commonly referred to as the 'Von Neumann bottleneck'. We can provide a Von Neumann processor with more cache, more RAM, or faster components but if original gains are to be made in CPU performance, then an influential inspection needs to take place of CPU configuration.

OPERATIONS AND OPERANDS OF COMPUTER HARDWARE INSTRUCTION

Instruction format

An **instruction format** specifies how the bits of an instruction are divided into different fields. It determines **how the CPU interprets an instruction** stored in memory. Instructions are stored as binary words, and each instruction is divided into **fields** that identify the **operation** and the **operands**.

Opcode	Mode	Address
---------------	-------------	----------------

Fig 3.4 Instruction fields

1. **Opcode field:** Defines the processor operation (add, subtract, complement, shift, etc.)

- A **group of bits** that defines the operation to be performed.
- It determines which hardware circuit (ALU, control, I/O) will be activated.
- Examples of processor operations:
 - Arithmetic → ADD, SUB, MUL, DIV.
 - Logical → AND, OR, NOT, XOR.
 - Shift/Rotate → SHL (Shift Left), SHR (Shift Right).

Control → JUMP, CALL, HALT.

Example: 1010 = ADD, 1011 = SUB.

2. Address field: Contains either a memory address or a register reference.

- It contains either:
- A **memory address** (to fetch/store data).
- A **register address** (to fetch/store data from CPU registers).
- Number of bits in this field decides the **addressing capacity** of the processor.

Example: 16 bits → $2^{16} = 65,536$ addresses.

Example: Address = 2000 → fetch data from memory location 2000.

3. Mode field: Defines the addressing technique used to obtain the operand.

- It specifies **how the operand or effective address is obtained**.
- It provides **flexibility** in instruction design.
- It allows multiple ways to access data:

Immediate (operand in instruction).

Direct (address given directly).

Indirect (address points to another address).

Register, Register Indirect, Indexed, Relative, etc.

4. Other special fields may appear in advanced processors (e.g., register fields, flag bits, immediate constants).

- **Register Field:** Identifies the register to be used.
- **Immediate Field:** Contains constant values.
- **Flag Field:** Used in conditional branching.

Types of Instruction Formats

There are mainly four types of instruction formats:

- Three address instructions
- Two address instructions
- One address instruction
- Zero address instructions

Three address instructions

- It contains **two source operands and one destination**.
- General form: ADD R1, R2, R3 → $R1 = R2 + R3$.
- Example: Cyber 170 (useful in high-level languages).
- It requires more bits (longer instruction).

The program in assembly language that evaluates $X = (A+B) * (C+D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

Add R1, A, B	$R1 \leftarrow M[A] + M[B]$
Add R2, C, D	$R2 \leftarrow M[C] + M[D]$
Mul X, R1, R2	$M[X] \leftarrow R1 * R2$

It is assumed that the computer has two processor registers, R1 and R2. The symbol M[A] denotes the operand at memory address symbolized by A.

Two address instructions

- One operand double as **source and destination**.
- General form: ADD R1, R2 $\rightarrow R1 = R1 + R2$.
- Example: Compact, reduces instruction length.

Two address instructions are the most common in commercial computers. The program to evaluate $X = (A+B) * (C+D)$ is as follows:

MOV R1, A	$R1 \leftarrow M[A]$
ADD R2, B	$R1 \leftarrow R1 + M[B]$
MOV R2, C	$R2 \leftarrow M[C]$
ADD R2, D	$R2 \leftarrow R2 + M[D]$
MUL R1, R2	$R1 \leftarrow R1 * R2$
MOV X, R1	$M[X] \leftarrow R1$

One address instruction

- It uses a single operand plus an **accumulator (ACC)** as an implicit second operand.
 - General form:

LOAD A
ADD B
STORE C
 - Example: ADD X $\rightarrow ACC = ACC + X$.
 - Common in early computers.
- The program to evaluate $X = (A+B) * (C+D)$ is

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result.

Zero address instructions

- No explicit operands; used in stack-based architectures.
- Operands are taken from the top of the stack.
- Example:

```
PUSH A
PUSH B
ADD
POP C
```

- It's very compact and simple.

The following program shows how $X = (A+B) * (C+D)$ will be written for a stack organized computer. (TOS stands for top of stack.)

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C+D)$
MUL		$TOS \leftarrow (C+D) * (A+B)$
POP	X	$M[X] \leftarrow TOS$

Operations of Computer Instructions

Computer instructions are broadly classified into three categories based on the function they perform: Arithmetic & Logical Operations, Data Transfer Operations, and Control Operations. Each of these is essential for program execution in a digital computer.

Types of Operation:

(a) Arithmetic Operations

- Perform mathematical calculations on integers.
- Operate on registers, not directly on memory (MIPS rule).

Examples:

1. Addition / Subtraction

- **add** \$s1, \$s2, \$s3 $\rightarrow$ \$s1 = \$s2 + \$s3
- **sub** \$s4, \$s5, \$s6 $\rightarrow$ \$s4 = \$s5 - \$s6
- Variants:
 - **add** (signed, overflow exception)
 - **addu** (unsigned, no exception)
 - **addi** (add immediate, register + constant).

2. Multiplication

- **mult** \$s1, \$s2 $\rightarrow$ product stored in Hi (upper 32 bits) and Lo (lower 32 bits).

3. Division

- **div** \$s1, \$s2 $\rightarrow$ quotient in Lo, remainder in Hi.

(b) Logical Operations

- Perform bit-by-bit manipulation of binary data.
- Useful for masking, testing, and bit operations.

Examples:

1. Bitwise AND / OR / NOR

- **and** \$s1, \$s2, \$s3 $\rightarrow$ each bit of \$s1 = \$s2 AND \$s3.
- **or** \$s1, \$s2, \$s3 $\rightarrow$ bitwise OR.
- **nor** \$s1, \$s2, \$s3 $\rightarrow$ NOT(OR).

2. Immediate Versions

- **andi** \$s1, \$s2, 0x0F $\rightarrow$ AND with a constant.

3. Shift Operations

- **sll** \$s1, \$s2, 4 $\rightarrow$ shift left by 4 bits (multiply by 2^4).
- **srl** \$s1, \$s2, 2 $\rightarrow$ shift right logical (divide by 2^2).

(c) Data Transfer Operations

- Move data between registers and memory.
- Because MIPS is load-store, arithmetic/logic only works on registers $\rightarrow$ you must load data first, store after.

Examples:

1. Load Instructions (memory $\rightarrow$ register)

- **lw** \$t0, 0(\$t1) $\rightarrow$ load word.
- **lb** \$t0, 0(\$t1) $\rightarrow$ load byte.
- **lhu** \$t0, 0(\$t1) $\rightarrow$ load halfword unsigned.

2. Store Instructions (register $\rightarrow$ memory)

- **sw** \$t0, 0(\$t1) $\rightarrow$ store word.

- **sb** \$t0, 0(\$t1) → store byte.
- **sh** \$t0, 0(\$t1) → store halfword.

Operation	Direction	MIPS Example	Description
Load	Memory → Register	lw \$s1, 8(\$s2)	Loads a word from memory into \$s1.
Store	Register → Memory	sw \$s1, 8(\$s2)	Stores the word from \$s1 to memory.
Load Byte	Memory → Register	lb \$t0, 0(\$t1)	Loads a single byte (8 bits).
Load Halfword	Memory → Register	lh \$t0, 0(\$t1)	Loads a halfword (16 bits).
Unsigned Load	Memory → Register	lbu, lhu	Fills upper bits with 0 instead of sign bit.
Atomic Load/Store	Sync between processors	ll, sc	Used in multiprocessor systems for safe updates.

(d) Control Flow (Decision-Making) Operations

- Change the normal sequential execution of instructions.
- Essential for loops, conditions, and function calls.

Examples:

1. Conditional Branches

- **beq** \$t1, \$t2, label → branch if equal.
- **bne** \$t1, \$t2, label → branch if not equal.
- (If condition false → continue sequential execution).

2. Unconditional Jumps

- **j label** → jump to instruction at *label*.
- **jr \$ra** → jump register (used for returning from functions).
- **jal label** → jump and link (store return address in \$ra).

Category	Purpose	Examples
Arithmetic	Perform math operations	add, sub, mult, div, addi
Logical	Bitwise manipulations	and, or, nor, sll, srl
Data Transfer	Move data between reg & memory	lw, sw, lb, sb
Control Flow	Alter execution path	beq, bne, j, jr, jal

Operands of Computer Instructions:

An operand is the data on which the CPU performs the operation specified by the opcode. It can be a value, a register, or a memory location.

1. Role:

- While the opcode tells what operation to perform, the operand tells on what data to perform it.

2. Types of Operands:

- **Immediate Operand** → actual constant value given in the instruction.
 - Example: **addi** \$t0, \$t1, 10 → here 10 is an immediate operand.
- **Register Operand** → stored inside CPU registers.
 - Example: **add** \$t0, \$t1, \$t2 → \$t1 and \$t2 are operands.
- **Memory Operand** → fetched from memory using an address.
 - Example: **lw** \$t0, 0(\$t1) → memory at address in \$t1 is the operand.

3. Number of Operands:

- Some instructions need one operand (e.g., **neg** \$t0).
- Others need two operands (e.g., **sub** \$t0, \$t1).
- Most arithmetic in MIPS uses three operands (destination + two sources)

Aspect	Opcode (Operation Code)	Operand
Definition	Field in the instruction that specifies the operation to be performed.	The data on which the operation is performed.
Role	Tells the CPU what to do (e.g., add, load, branch).	Tells the CPU what data to use (value, register, or memory).
Form	Always a binary code inside the instruction format.	Can be a register value, memory location, or immediate constant.
Example	In add \$s0, \$s1, \$s2 → add is the opcode.	In add \$s0, \$s1, \$s2 → \$s1 and \$s2 are operands.
Dependency	Without operands, opcode knows <i>what</i> to do but not <i>on what</i> .	Without opcode, operands are just meaningless data.

INSTRUCTION SET ARCHITECTURE (ISA)

- The **Instruction Set Architecture (ISA)** is the abstract interface between the hardware and the lowest-level software. It provides all the necessary information for writing correct binary machine

language programs — including details about **instructions, registers, and memory access**.

- The ISA controls **how memory locations are addressed** and **how operations are performed** within a computer system.
- The ISA provides essential details about:
 - **Instructions**
 - **Registers**
 - **Memory access**
 - **Input/Output (I/O)**.

The foundation of modern computer architecture rests on the stored-program concept, which states that:

1. **Instructions are represented as numbers**, allowing them to be processed like data.
2. **Programs are stored in memory and accessed like data**, enabling the computer to fetch, decode, and execute instructions sequentially or conditionally

Memory Locations

The ISA defines the memory system as the main storage area where both programs and data are kept while executing.

- Memory can be visualized as a large one-dimensional array of storage cells.
- Each cell in this array has an **address**, which acts as an **index** pointing to the data stored there.
- Each cell (location) can store a fixed number of bits — commonly 8 bits = 1 bytes.
- Memory consists of millions of storage cells, each storing 1 bit (0 or 1).
- A single bit is too small to handle individually, so bits are grouped into larger units called words.
- A word is the natural unit of data used by the processor — typically 16, 32, or 64 bits.

Example:

If a system has a 32-bit word size, one word = 4 bytes.

If memory starts at address 0, the 100th location will have address 99.

Address → 0 1 2 3 4...

Data → 25 18 42 19 77 ...

- During execution:
 - Data is **fetches from memory to registers** for processing.
 - Results are then **stored back to memory** after processing

Memory Addresses

- Each memory location has a unique numerical address used by the CPU to access it.
 - The range of addresses depends on the number of address lines (n). For n address lines, total addressable locations = 2^n .
 - Example:
If a memory has 20 address lines, it can address $2^{20} = 1,048,576$ (1 MB) locations.
 - Each word or byte has a unique address to allow storage and retrieval.
 - Addresses usually range from 0 to $2^k - 1$, where k = number of address bits.
 - Address space: Total number of addressable locations.
- Examples:
- 24-bit address: $2^{24} = 16,777,216$ locations $\approx 16\text{M}$
 - 32-bit address: $2^{32} \approx 4,294,967,296$ locations $\approx 4\text{G}$

Addressing in Words and Bytes

Most modern systems are **byte-addressable**, meaning:

- Each **byte** has its own unique address.
- Words are formed by grouping consecutive bytes.

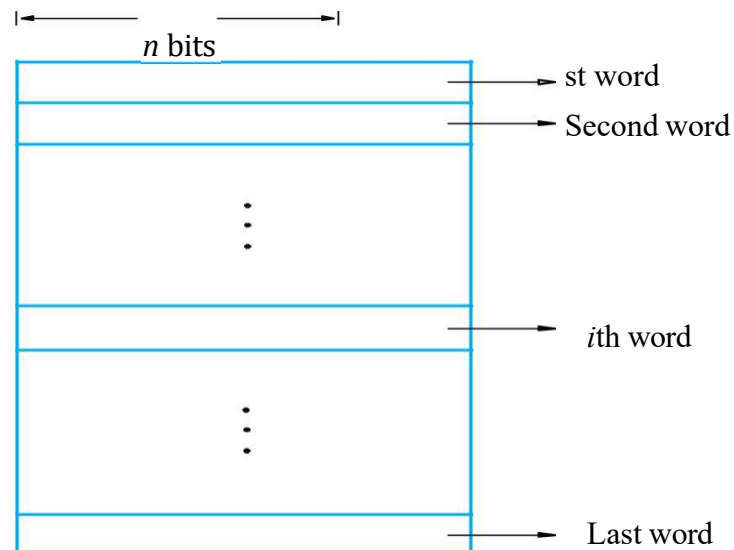


Fig 3.7 Memory words

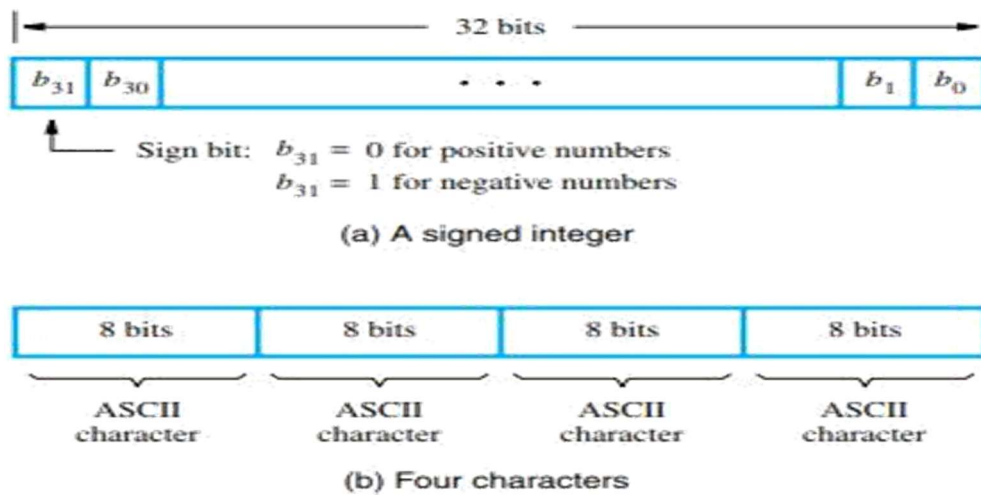


Fig 3.8 Examples of encoded information in a 32-bit word

If the processor's word length = 32 bits (4 bytes):

- Word 0 → bytes 0–3
- Word 1 → bytes 4–7
- Word 2 → bytes 8–11

So, **successive word addresses differ by 4**

Byte Addressability

- It is impractical to assign separate addresses to individual bits in memory — it would require too many addresses and make memory management complex. Hence, modern computers assign addresses to bytes rather than bits.
- When a word consists of multiple bytes (for example, a 32-bit word = 4 bytes), the **order** in which these bytes are stored in memory can differ between computer architectures.
- A memory is **byte-addressable** if **each byte** has a separate address.

Endianness:

The order in which bytes of a word are stored in memory depends on the architecture.

i. Big-Endian Format

- The **most significant byte (MSB)** — i.e., the leftmost or “highest value” byte — is stored at the **lowest memory address**.
- The bytes are stored in **decreasing significance** as addresses increase.
- Used by systems like Motorola processors.

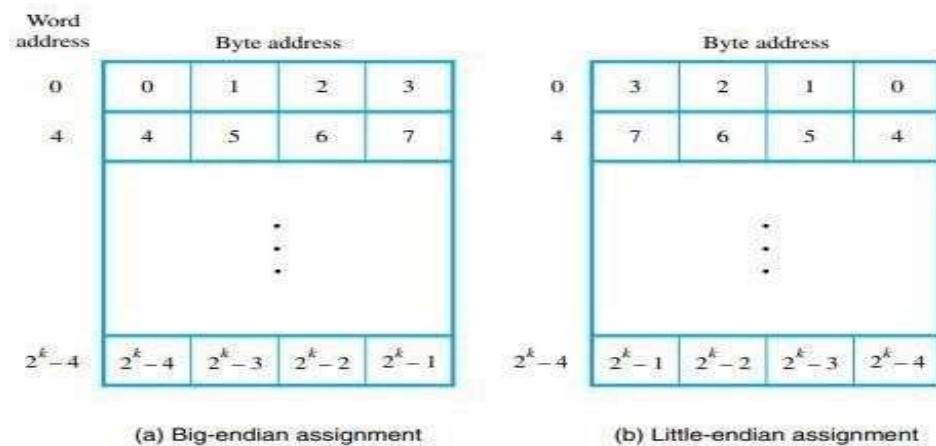


Fig 3.9 Byte and word addressing.

ii. Little-Endian Format

- The **least significant byte (LSB)** — i.e., the rightmost or “lowest value” byte — is stored at the **lowest memory address**.
- Bytes are stored in **increasing significance** as addresses increase.
- Used by Intel processors.

Example (32-bit word = 0x12345678):

Address	Big-Endian	Little-Endian
1000	12	78
1001	34	56
1002	56	34
1003	78	12

Memory operations

In a digital computer, both program instructions and data operands are stored in memory. For program execution, the processor performs two main memory interactions:

1. Fetching instructions from memory to the processor.
2. Reading/Writing operands and results between processor registers and memory.

Thus, two fundamental operations are defined:

- Read Operation
- Write Operation

1. READ OPERATION

To transfer the contents of a specific memory location into the processor, without changing

the memory contents.

Typical control signal setup:

Memory Enable = 1, Read/Write = 1 → Read Operation.

Process:

1. The processor sends the binary address of the desired word to the address lines of memory.
2. It activates the Read control signal.
3. The memory decodes the address and places the data word stored in that location onto the data output lines.
4. The processor receives the data and loads it into a register.

2. WRITE OPERATION

To transfer a data word from the processor into a specific memory location, replacing its old contents.

Typical control signal setup:

Memory Enable = 1, Read/Write = 0 → Write Operation.

Process:

1. The processor sends the address of the target word to the address lines.
2. It sends the data to be written onto the data input lines.
3. The processor activates the Write control signal.
4. The memory stores the new data word in the addressed location.

Memory operations Notation

- To describe the operations inside a computer, we use special notations that indicate how data moves between registers, memory, and input/output units.
- Two common ways to represent these operations are:
 1. Register Transfer Notation (RTN)
 2. Assembly Language Notation

Processor and Register Naming

- Processor registers are typically denoted as R0, R1, R2, ... or sometimes R5, etc.
- Input/Output subsystem registers may have descriptive names such as DATAIN, OUTSTATUS, or DATAOUT.
- These symbolic names make it easier to express data movement and computation.

Register Transfer Notation (RTN)

It is a symbolic method to describe the transfer of data and operations performed by the CPU at the register level.

The general syntax is:

Destination ← Source

- The left-hand side (LHS) represents the destination (where the data will be stored).
- The right-hand side (RHS) represents the source (the value or data to be transferred).

- Square brackets [] around a name denote contents of that location.

Examples

1. Simple Data Transfer

$R2 \leftarrow [LOC]$

The contents of memory location LOC are copied into register R2.
(Only R2 changes; LOC remains unchanged.)

2. Arithmetic Operation

$R4 \leftarrow [R2] + [R3]$

Add the contents of registers R2 and R3, and store the result in R4.

3. Meaning of Transfer

The word “transfer” (or “move”) actually means copy —
the destination gets the new value,
the source remains unchanged.

Key Points of RTN

Feature	Description
Symbol [X]	Contents of location X
$\leftarrow$	Assignment / Transfer symbol
LHS	Destination (receives data)
RHS	Source (provides data)
Example	$R1 \leftarrow [R2] + [R3]$ means add contents of R2 and R3 and store in R1

Assembly Language Notation

- An assembly language uses mnemonic symbols (short abbreviations) to represent machine-level instructions.
- It provides a human-readable format to describe the same operations expressed in RTN.

Example 1 — Data Transfer

RTN: **$R2 \leftarrow [LOC]$**

Assembly Language: **Load R2, LOC**

Meaning: Copy contents of memory location LOC into register R2.
(LOC is unchanged; R2 is overwritten.)

Example 2 — Arithmetic Operation

RTN: **$R4 \leftarrow [R2] + [R3]$**

Assembly Language: **Add R4, R2, R3**

Meaning: Add the contents of registers R2 and R3, and store the result in R4.
Here:

- $R2, R3 \rightarrow$ Source operands

- R4 → Destination register

Mnemonics in Assembly

Mnemonics are short, symbolic names for operations:

Operation	Mnemonic Example
Load	LD
Store	ST or STR
Add	ADD
Subtract	SUB
Move	MOV

Example:

LD R2, LOC; Load data from memory to register

ST R3, RESULT; Store register contents into memory

ADD R1, R2, R3; Add R2 and R3, store in R1

INSTRUCTION AND INSTRUCTION SEQUENCING

Instruction:

- A binary-encoded command that tells the CPU to perform a specific operation (e.g., add, load, branch).
- It is a binary command that directs the processor to perform a specific operation — such as moving data, performing arithmetic, or controlling the program's flow.
- A collection of such instructions is known as the Instruction Set Architecture (ISA).

Instruction Set: The complete collection of instructions supported by a CPU.

Instruction Execution Cycle:

1. Fetch Phase: The instruction is fetched from the memory location pointed to by the Program Counter (PC) and stored in the Instruction Register (IR).
2. Decode Phase: The instruction in IR is decoded to identify the operation and operands.
3. Execute Phase: The operation is carried out using CPU registers or memory.
4. PC Update: The Program Counter is updated to the next instruction address.

Structure of an Instruction:

Each instruction in memory is represented as a **binary word** divided into fields:

- **Opcode (Operation Code):** Specifies the operation to be performed (e.g., ADD, LOAD, STORE).
- **Operands:** Identify data or addresses involved in the operation.
- **Addressing Mode:** Describes how to interpret or locate the operands.
- Instruction format depends on the computer architecture (e.g., MIPS uses 32-bit fixed-length instructions).

Instruction Sequencing: It refers to the order in which instructions are fetched, decoded, and

executed by the processor, under the control of the Program Counter (PC) and control unit.

Sequential Execution

- Normally, instructions are executed in sequence.
- After executing an instruction, the **Program Counter (PC)** is incremented to point to the next instruction ($PC = PC + 4$ in 32-bit systems).

PC → Fetch Instruction → Decode → Execute → PC + 1 → Repeat

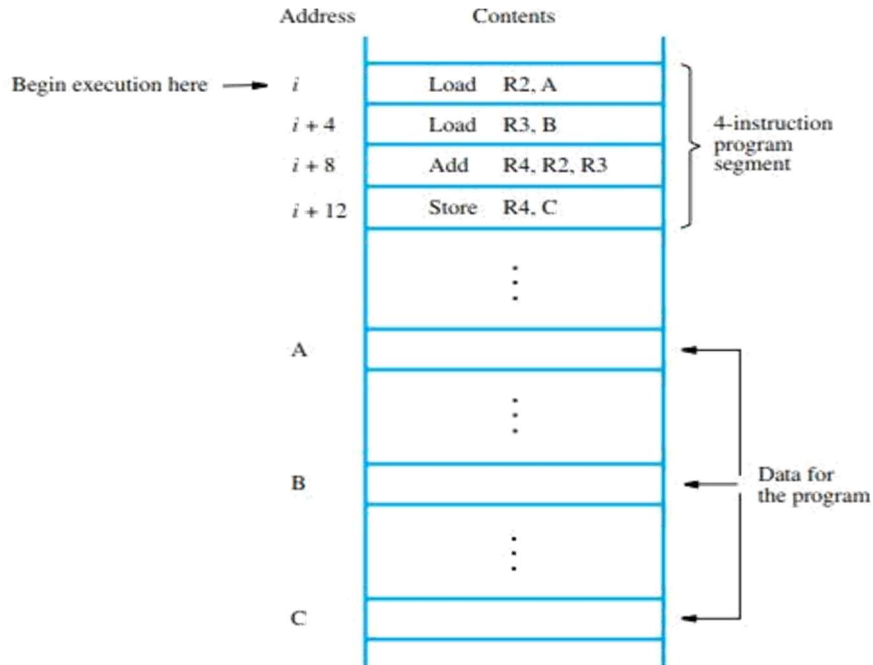


Fig 3.10 A program for $C \leftarrow [A] + [B]$.

Address	Instruction	Operation Description
i	Load R1, A	Copy the contents of memory location A into register R1
$i + 4$	Load R2, B	Copy the contents of memory location B into register R2
$i + 8$	Add R3, R1, R2	Add the contents of R1 and R2, place the sum in R3
$i + 12$	Store R3, C	Store the result from R3 into memory location C

Non-sequential Execution (BRANCHING)

- Some instructions **change the normal flow** of execution.
- These instructions modify the PC to jump to another address.

- Examples include:
 - **Branch** → changes PC based on a condition.
 - **Jump** → unconditional control transfer.
 - **Call and Return** → for subroutines.
- Branching allows the CPU to **change the sequence of instruction execution** by modifying the Program Counter (PC) to a new address.
- Branch instruction modifies the Program Counter (PC) so that:
 - **PC ← Branch Target Address**
- This means the next instruction fetched will be from the branch target, not the next sequential address.
 - If the condition is true, the branch is taken → control jumps to target.
 - If false, the branch is not taken → execution continues sequentially.

Example Task:

Add a list of n numbers stored in memory locations NUM1, NUM2, ... NUMn.

Instead of writing multiple LOAD and ADD instructions, we can use a **loop with a branch** instruction.

SEQUENTIAL PROGRAM WITHOUT BRANCHING

```

LOAD R1, NUM1; Load first number into R1
LOAD R2, NUM2; Load second number into R2
ADD R1, R1, R2; R1 = NUM1 + NUM2
LOAD R2, NUM3
ADD R1, R1, R2; R1 = (NUM1 + NUM2 + NUM3)
LOAD R2, NUM4
ADD R1, R1, R2; R1 = (NUM1 + NUM2 + NUM3 + NUM4)
...
LOAD R2, NUMn
ADD R1, R1, R2; Final sum in R1
STORE R1, SUM; Store result in memory

```

- Each number is explicitly loaded and added to the accumulator (or a register).
- The same instruction pattern is repeated for every number.
- The number of instructions increases with n .
- This approach is **long and inefficient**.

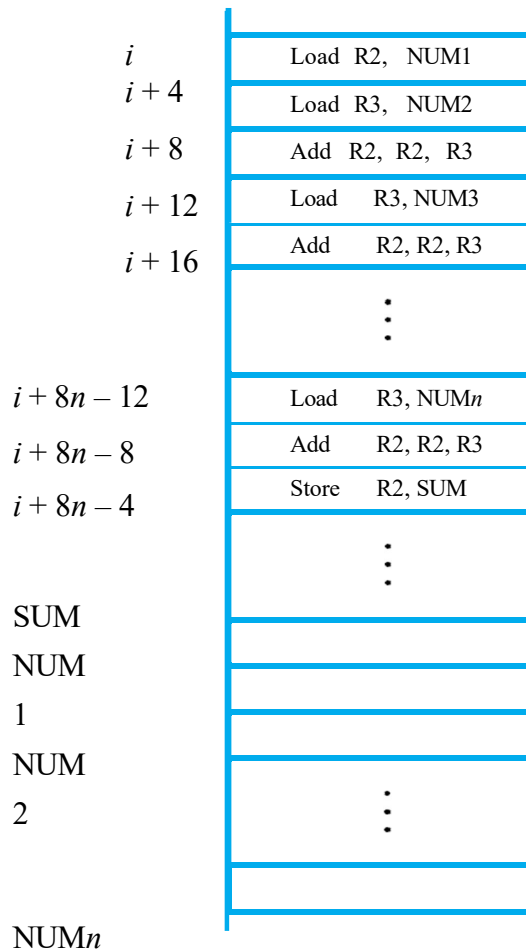


Fig 3.11 A program for adding n numbers.

PROGRAM WITH BRANCHING

LOAD R2, N; Load the number of elements (counter)
 CLEAR R3; Initialize SUM register to 0
 LOAD R4, NUM1; Load base address of NUM1
LOOP: LOAD R5, (R4); Load current number into R5
 ADD R3, R3, R5; Add current number to SUM (R3)
 ADDI R4, R4, #4; Move to next memory address (word = 4 bytes)
 SUB R2, R2, #1; Decrement counter
 BGT R2, R0, LOOP; Branch to LOOP if R2 > 0
 STORE R3, SUM; Store final result in memory
HALT

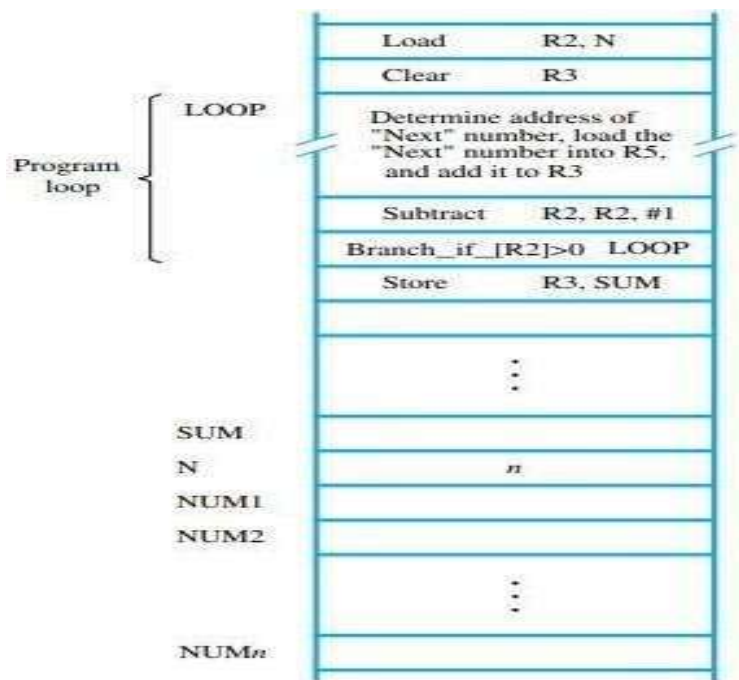


Fig 3.12 Using a loop to add n numbers

- A **loop** executes the same block of instructions multiple times using a **conditional branch**.
- The loop continues **while a condition is true** — for example, while a counter register value > 0 .
- This greatly reduces program size and improves flexibility.

Type	Condition	Example Instruction	Description
Unconditional Branch	Always true	JMP TARGET	Jumps always
Conditional (Equality)	$=, \neq$	BEQ, BNE	Tests for equality
Relational Conditional	$>, <, \geq, \leq$	BGT, BLT, BGE, BLE	Tests magnitude
Flag-Based	Status bits	BZ, BNZ, BC, BNC	Tests condition flags
Counter-Based	Loop control	BGT R2, R0, LOOP	For iteration control

STACKS

- A stack is a data structure that stores a collection of elements with a restriction that insertions and deletions can occur only at one end, called the top of the stack. The opposite end is called the bottom.
- It follows the LIFO (Last-In, First-Out) principle — the last element inserted into the stack is the first one to be removed.

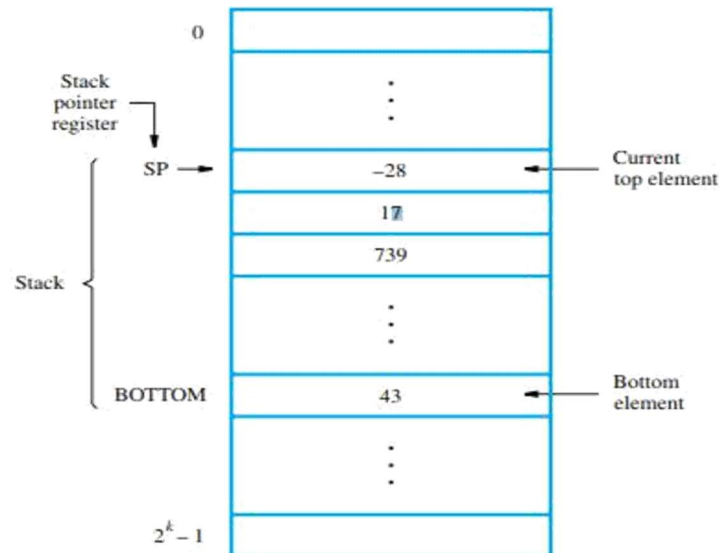


Fig 3.13 A stack of words in the memory.

Stack Structure

- The figure shows a stack of words stored in consecutive memory locations.
- Each rectangular block represents one word (4 bytes) in memory.
- Memory addresses decrease from bottom to top, meaning the stack grows downward.

Stack Pointer (SP)

- The Stack Pointer (SP) register holds the address of the top element in the stack.
- When an element is pushed, the SP is decremented.
- When an element is popped, the SP is incremented.

Term	Meaning
Push	Operation to insert (add) a new element on the top of the stack
Pop	Operation to remove (delete) the topmost

Term	Meaning
	element from the stack
Top	Refers to the most recently added item (accessible element)
Bottom	The first inserted element, which remains until all elements above are removed
Stack Pointer (SP)	A special processor register that holds the address of the top element in the stack

Stack Operations

1. PUSH Operation

To push a new item from register Rj onto the stack:

Subtract SP, SP, #4

Store Rj, (SP)

→ Decrements SP and places the content of register Rj on the stack.

2. POP Operation

Load Rj, (SP)

Add SP, SP, #4

→ Loads the top of the stack into register Rj and increments SP to the next element.

SUBROUTINES

A subroutine is a sequence of instructions that can be called from various points in a program to perform a particular operation and then return control back to the calling program.

Examples:

- Computing a mathematical function
- Searching or sorting a list
- Handling input/output operations

Subroutine Execution Cycle

1. Calling a Subroutine (Call Instruction)

- When the main program needs to execute the subroutine:
- The Call instruction is executed.
- The address of the next instruction (the return address) — which is currently held in the Program Counter (PC) — must be saved.
- The processor branches to the subroutine's starting address.
- This ensures that, after completing the subroutine, the processor can return to the correct place in the main program.

2. Returning from a Subroutine (Return Instruction)

- After executing the subroutine:

- The Return instruction is executed.
- The processor restores the saved return address (usually from a register or the stack).
- Control returns to the instruction immediately following the Call instruction.

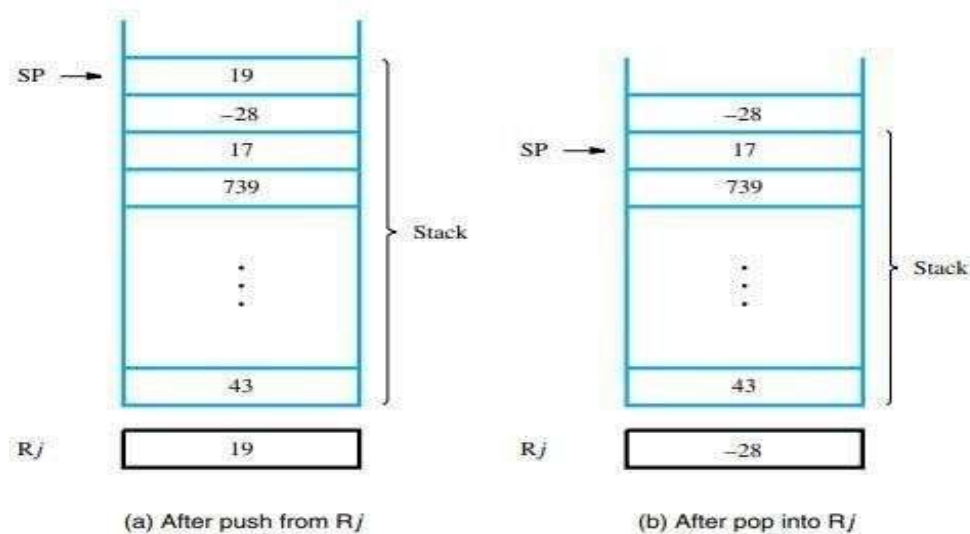


Fig 3.14 Effect of stack operations on the stack in Fig 3.13.

Subroutine Linkage

The method of connecting the calling program and the subroutine — that is, saving and restoring the return address — is called the subroutine linkage method.

There are two main approaches:

(i) Link Register Method

- The return address is stored in a dedicated register called the **Link Register (LR)**.
- When the subroutine completes, the **Return instruction** branches indirectly to the address stored in the link register.

Operations performed:

Call instruction:

Store PC → Link Register

Branch to Subroutine Address

Return instruction:

Branch to address in Link Register

(ii) Stack-Based Method (Advanced Linkage)

- For **nested subroutine calls** or **recursive functions**, multiple return addresses must be stored.

- This is achieved by **saving the return address on the stack.**

Process:

Call instruction:

PUSH PC onto Stack

Branch to Subroutine Address

Return instruction:

POP Top of Stack into PC

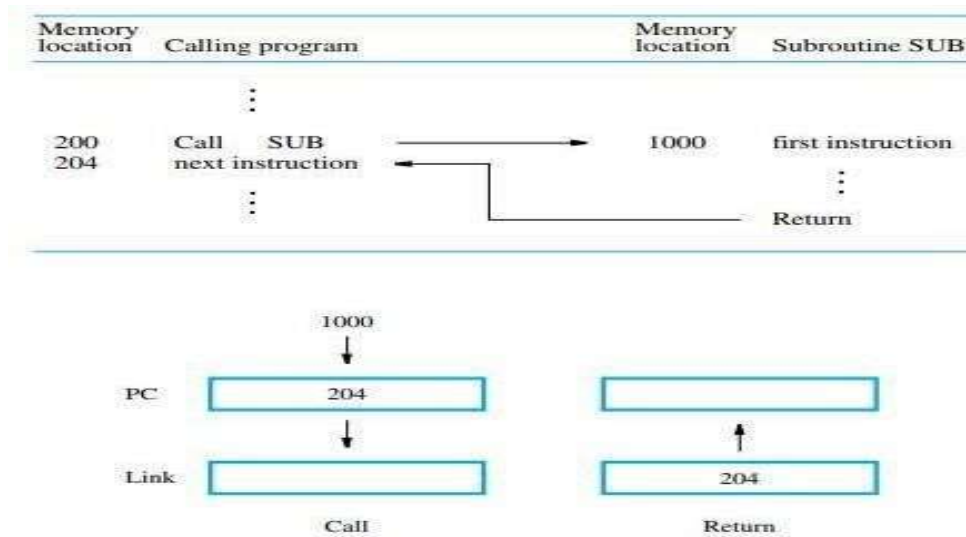


Fig 3.15 Subroutine linkage using a link register.

ADDRESSING MODES

- An addressing mode defines how the effective address (EA) of an operand is specified in an instruction.
- In other words, it tells the microprocessor how to locate the data required for execution — whether it is stored in a register, in memory, or embedded in the instruction itself.
- Every instruction must specify:
 1. The operation to perform,
 2. The address of the source operand, and
 3. The address of the destination result.

TYPES OF ADDRESSING MODES:

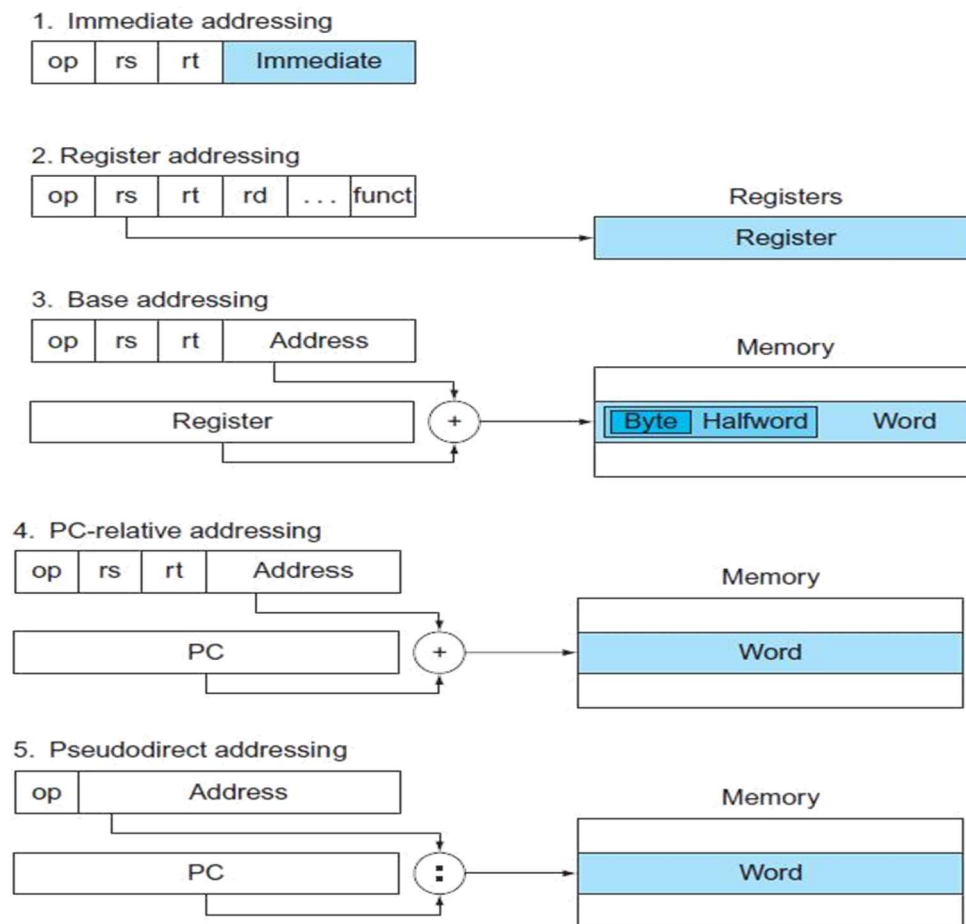


Fig3.16 Various Addressing Modes

a. Implied Addressing Mode

- Operand is implicitly specified by the instruction.
- Used for single-operand instructions (e.g., accumulator operations).
- Example:
CMA; Complement Accumulator (operand implied)
- The operand (accumulator) is implied in the operation.

b. Immediate Addressing Mode

- Operand value is directly specified in the instruction.
- No memory reference needed.

- Used for: initialization, constants.
- Example:
- `ADD #10`; Add constant 10 to accumulator
EA = value within the instruction.

c. Direct (Absolute) Addressing Mode

- The address field of the instruction gives the exact memory address of the operand.
- Example:
- `LDA A`; Load contents of memory location A into accumulator
EA = address field in instruction.

d. Indirect Addressing Mode

- The address field gives the location of the address of the operand.
- Requires two memory accesses — one to fetch the effective address and another to fetch the operand.
- Example:
`ADD (A)`; Add operand pointed by address in memory location A
EA = contents of the address field's memory location.

e. Register Addressing Mode

- Operand is stored in a register.
- The register name is part of the instruction.
- Example:
`ADD R1, R2`; Add contents of R1 to R2
EA = register contents.

f. Register Indirect Addressing Mode

- A register contains the memory address of the operand.
- Example:
- `ADD (R2)`; Add operand from memory location pointed by R2
EA = contents of R2.

g. Auto-Increment / Auto-Decrement Addressing Mode

- Auto-Increment: operand fetched from address in register, then register increments automatically.
Syntax: (Ri)+
- Auto-Decrement: register decremented first, then operand fetched.
Syntax: -(Ri)
- Examples:
ADD (R1) +; Use operand, then increment R1
LDA -(R1); Decrement R1, then load operand

h. Relative Addressing Mode

- Operand address = current PC + displacement value from instruction.
- Used in branching.
- Example:
If PC = 826 and displacement = 24 →
EA = 826 + 24 = 850
Branch target is 24 bytes ahead of the next instruction.

i. Indexed Addressing Mode

- Operand address = base address + index value (index register).
- Common for array data access.
- Example:
MOV AL, DS: disp [SI]; Move from address (disp + SI)
EA = Address field + [Index Register]

j. Base Register Addressing Mode

- Operand address = contents of base register + displacement.
- Facilitates program relocation (used in multiprogramming systems).
- Example:
MOV A, 10(RB); Access memory at Base + 10
EA = [Base Register] + Displacement

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R_i	$EA = R_i$
Absolute	LOC	$EA = LOC$
Register indirect	(R_i)	$EA = [R_i]$
Index	$X(R_i)$	$EA = [R_i] + X$
Base with index	(R_i, R_j)	$EA = [R_i] + [R_j]$

Table 3.1 RISC-type addressing modes.

Mode	Effective Address (EA)	Example
Implied	Operand implied	CMA
Immediate	Operand = constant	ADD #5
Direct	$EA = \text{address field}$	LDA A
Indirect	$EA = [\text{address field}]$	ADD (A)
Register	$EA = R_i$	ADD R1, R2
Register Indirect	$EA = [R_i]$	ADD (R2)
Auto-Increment	$EA = [R_i]; R_i = R_i + 1$	ADD (R1) +
Auto-Decrement	$R_i = R_i - 1; EA = [R_i]$	LDA -(R1)
Relative	$EA = PC + \text{displacement}$	BRANCH +24
Indexed	$EA = A + [\text{Index Reg}]$	MOV AL, DS: [SI]
Base Register	$EA = [\text{Base Reg}] + \text{displacement}$	MOV BX, 10(RB)

Purpose of Addressing Modes

Computers use addressing modes to:

- Provide programming flexibility, such as using pointers, counters, and indexing.
- Simplify program relocation (useful in multiprogramming systems).
- Reduce instruction length, minimizing the number of bits needed to specify addresses.

